

Devoir maison n°3

À rendre le lundi 11/10

Pour ce DM, vous utiliserez les fichiers texte disponible sur le dépôt <https://github.com/nathaniel-carre/MPI-LLG>, dans le dossier `Devoirs/DM/DM3_Kosaraju`.

Pour ce devoir, vous devrez rendre une copie papier, contenant la rédaction des questions qui ne sont pas de la programmation, ainsi qu'un fichier de code source écrit en C, contenant les fonctions et les tests que vous aurez écrit.

Il est demandé que le fichier source puisse se compiler correctement. Tout code erroné encore présent dans le fichier devra être commenté. Vous nommerez vos fichiers au format `DM3_NOM_Prenom.c`, par exemple `DM3_CARRÉ_Nathaniel.c`.

Les fichiers devront être déposés sur le casier numérique de l'ENT (via Mon Drive Monlycee).

La copie papier devra contenir au moins une demi-page complète blanche, à destination du correcteur, pour la rédaction des commentaires lors de la correction du code. Le code source devra inclure des commentaires de code expliquant le fonctionnement des différentes parties du code. Il devra également contenir des tests des différentes fonctions écrites.

Les tests devront être fait avec l'option de compilation `-fsanitize=address`, et on s'assurera de l'absence de fuites mémoire.

1 Lecture de fichiers

Dans l'ensemble du devoir, on travaille avec des graphes, orientés ou non, implémentés par tableaux de listes d'adjacence.

Chacun des fichiers `nonorienté1` à `nonorienté8` et `orienté1` à `orienté8` contient la description d'un graphe non orienté ou orienté. Le format de ces fichiers est le suivant :

- la première ligne contient le nombre de sommets;
- chaque ligne après la première correspond à un des sommets s (en commençant la numérotation des sommets à 0) et contient le degré du sommet, $\deg(s)$, suivi de $\deg(s)$ sommets qui sont les voisins de s .

Voici, par exemple, les 4 premières lignes de `nonorienté1` :

```
10
1 1
2 0 7
3 9 8 6
```

Cela signifie que le graphe décrit contient 10 sommets, que le sommet 0 est de degré 1 et a comme voisin le sommet 1, que le sommet 1 est de degré 2 et a comme voisin les sommets 0 et 7, ...

Question 1 Représenter graphiquement les graphes des fichiers `nonorienté1`, `nonorienté2`, `orienté1` et `orienté2`, en essayant de limiter les croisements d'arêtes. On les utilisera pour vérifier les différentes fonctions.

On souhaite, dans un premier temps, lire ces fichiers pour les transcrire en un type de graphe implémenté en C. On utilise le type suivant :

```
struct Graphe {
    int n;
    int* deg;
    int** adj;
};

typedef struct Graphe graphe;
```

de telle sorte que si $G = (S, A)$ est un graphe implémenté par l'objet `G` de type `graphe`, alors :

- $S = \llbracket 0, |S| - 1 \rrbracket$ et $|S| = G.n$;
- `G.deg` est un tableau d'entiers de taille `G.n` tel que pour $s \in S$, `G.deg[s]` = $\deg(s)$;
- `G.adj` est un tableau de taille `G.n` tel que pour $s \in S$, `G.adj[s]` est un tableau de taille `G.deg[s]` contenant les voisins de s dans un ordre arbitraire.

On rappelle que les fonctions suivantes permettent d'effectuer de la lecture de fichier et sont utilisables avec l'inclusion de l'entête `stdio.h`. On pourra se référer à une documentation pour plus de détails :

- `fopen` ouvre un fichier texte et renvoie un pointeur vers le canal de lecture, de type `FILE*` ; par exemple, l'appel `fopen("/MPI/DM3/oriente1", "r")` permet d'ouvrir le fichier `oriente1` en mode lecture ;
- `fscanf` permet de lire la ligne suivante du fichier selon un certain formatage ; par exemple, si `canal` est un canal de lecture et si `a`, `b` et `c` sont des variables entières, alors `fscanf(canal, "%d %d %d", &a, &b, &c)` permet de lire une ligne contenant trois entiers et de stocker leurs valeurs dans les variables `a`, `b` et `c`. La fonction renvoie le nombre de valeurs affectées si la lecture s'est déroulée normalement (donc 3 dans l'exemple précédent), et renvoie `EOF` si le canal était à la fin du fichier ;
- `fclose` ferme un canal de lecture. Par exemple, `fclose(canal)` permet de fermer `canal`.

Question 2 Écrire une fonction `graphe lire_graphe(const char* nom_fichier)` qui prend en argument un nom de fichier correspondant à un graphe et renvoie le graphe correspondant.

Question 3 Écrire une fonction `void afficher_tableau(int* tab, int n)` qui prend en argument un tableau d'entiers et sa taille et l'affiche.

On utilisera cette fonction pour vérifier le bon déroulement d'un appel à la fonction précédente.

Question 4 Écrire une fonction `void liberer_graphe(graphe G)` qui libère l'espace mémoire utilisé par un graphe donné en argument.

2 Graphes non orientés

Question 5 Écrire une fonction `int nb_cc(graphe G)` qui compte le nombre de composantes connexes d'un graphe donné en argument.

Question 6 Écrire une fonction `bool acyclique(graphe G)` qui teste si un graphe **non orienté** $G = (S, A)$ donné en argument est acyclique ou non. Cette fonction devra avoir une complexité linéaire en $|S|$ (et pas seulement en $|S| + |A|$) et on justifiera cette complexité. On pourra utiliser ou non la fonction précédente.

Question 7 Parmi les graphe non orientés fournis, lesquels sont acycliques ?

3 Graphes orientés

Toutes les fonctions de cette partie sur un graphe $G = (S, A)$ devront avoir une complexité en $\mathcal{O}(|S| + |A|)$.

3.1 DAG

Question 8 Écrire une fonction `int* num_post(graphe G)` qui prend en argument un graphe $G = (S, A)$ et renvoie un tableau d'entiers `post` de taille $|S|$ tel que pour $s \in S$, `post[s]` est le numéro postfixé de s dans un parcours en profondeur de G .

Question 9 En déduire une fonction `bool dag(graphe G)` qui teste si un graphe **orienté** $G = (S, A)$ donné en argument est un DAG ou non.

Question 10 Parmi les graphes orientés fournis, lesquels sont des DAG ?

3.2 Algorithme de Kosaraju

Question 11 Écrire une fonction `int* inverse_post(int* post, int n)` qui prend en argument un tableau `post` correspondant aux numéros postfixes d'un parcours en profondeur d'un graphe et le nombre de sommets `n` du graphe et renvoie un tableau de taille `n` correspondant à l'ordre inverse du parcours postfixe du graphe. Cette fonction devra avoir une complexité linéaire en `n` mais on ne demande pas de le justifier.

On notera que le graphe du parcours n'est pas donné en argument de cette fonction.

Question 12 Écrire une fonction `graphe transpose(graphe G)` qui prend en argument un graphe orienté G et renvoie son transposé G^T .

Question 13 En déduire une fonction `int* kosaraju(graphe G)` qui prend en argument un graphe orienté $G = (S, A)$ et renvoie un tableau `cfc` de taille `|S|` tel que pour $s \in S$, `cfc[s]` est le numéro de la composante fortement connexe de s , en supposant que les numéros sont des entiers consécutifs commençant à zéro.

Question 14 En déduire une fonction `int nb_cfc(graphe G)` qui prend en argument un graphe orienté et renvoie son nombre de composantes fortement connexes.

Question 15 Quels sont les nombres de composantes fortement connexes des graphes orientés fournis ?